CAPSTONE PROJECT REPORT

CloudLens

DevOps Dashboard for Resource and Cost Monitoring

Multi-cloud (AWS · Azure · GCP) cost aggregation, trend analysis and alerting

Submitted by: [Student Name]
Enrolment / Roll No.: [Number]
Programme: [Programme name]
Institute: [Institute name]
Guide / Mentor: [Mentor name]

October 2026

Abstract

DevOps teams that operate on several public clouds see their spending in three separate consoles, each with a different vocabulary, billing delay and level of detail. Without one consolidated view it is hard to know where money is going, to spot a runaway workload before the invoice arrives, or to prove that infrastructure is right-sized.

This project delivers **CloudLens**, a containerised DevOps dashboard that aggregates daily cost and resource-utilisation data from **AWS Cost Explorer, Azure Cost Management and the GCP Cloud Billing export**, normalises it into one PostgreSQL schema, and presents real-time and historical views in a custom React.js dashboard. Users can filter and drill down by provider, environment, resource type and date range; analyse month-over-month trends and anomalies; and define their own alert rules for cost thresholds, budget forecasts, cost anomalies and over- or under-utilised resources. Alerts flow through **Prometheus and Alertmanager** to **Slack and Microsoft Teams**.

A rule-based optimisation engine converts the monitoring data into concrete savings actions (stop idle machines, rightsize, schedule non-production workloads, tier storage). On the demonstration dataset it identifies about USD 9.2K of monthly savings, roughly 8.5 % of spend. The platform itself runs on one t3.small EC2 instance for about USD 21 per month, through careful API usage and data-retention policies.

The solution is built with Python (FastAPI, SQLAlchemy, APScheduler), PostgreSQL 16, React 18 and Recharts, packaged as six Docker containers, and deployable to AWS EC2 with the provided scripts. It is verified by 77 backend tests (run against both SQLite and PostgreSQL), frontend unit tests, and an end-to-end test of the full Prometheus → Alertmanager → Slack/Teams alert pipeline.

Keywords: FinOps, multi-cloud, cost monitoring, DevOps dashboard, Prometheus, Alertmanager, FastAPI, React, PostgreSQL, Docker.

Table of Contents

1. Introduction 3
1.1 Background 3
1.2 Problem statement 3
1.3 Project goals 3
1.4 Scope 3
1.5 Report organisation 3
2. Existing Solutions and Gap Analysis 4
3. Requirements 5
3.1 Functional requirements 5
3.2 Non-functional requirements 5
4. System Architecture 6
4.1 Data flow 6
4.2 Technology choices 6
5. Design 8
5.1 Unified cost schema 8
5.2 Database tables 8
5.3 API design 8
5.4 Module structure 8
6. Implementation 10
6.1 Cloud integration (Sprint 1) 10
6.2 Aggregation and storage (Sprint 2) 10
6.3 Real-time dashboard (Sprint 3) 10
6.4 Historical analysis (Sprint 4) 11
6.5 Resources and drill-down 12
6.6 Authentication and roles 13
6.7 Currency and theme 13
7. Alerting and Notifications (Sprint 5) 14
8. Cost Optimisation 16
8.1 Savings for the monitored clouds 16
8.2 Keeping the platform itself cheap 16
9. Testing 17
10. Deployment 18
11. Results 19
12. Conclusion and Future Scope 25
12.1 Conclusion 25
12.2 Learnings 25
12.3 Future scope 25
References 26
Appendix A: Repository and Documentation Map 27
Appendix B: Key Configuration 28

1. Introduction

1.1 Background

Public cloud adoption lets teams provision infrastructure in minutes, but the pay-as-you-go model also means costs can grow silently. Organisations commonly use more than one provider, for example AWS for core services, Azure for Microsoft workloads and GCP for analytics. Each provider offers its own cost tool, but none gives a consolidated, comparable view across clouds, and native alerting is limited to per-provider budgets.

1.2 Problem statement

DevOps teams struggle to monitor resource usage and manage costs across multiple cloud environments (AWS, Azure and GCP). Without clear visibility into resource allocation and spending, optimising cloud infrastructure is difficult, which leads to unnecessary costs and inefficiencies. The project aims to develop a centralised dashboard that aggregates data from the different cloud providers, lets teams track resource utilisation and cost trends, set cost thresholds, and receive alerts for anomalies.

1.3 Project goals

1. Aggregate resource-utilisation and cost data from AWS, Azure and GCP into a single dashboard.
2. Provide real-time and historical visualisation of resource usage and costs to help teams identify trends and optimise resources.
3. Implement customisable alerting for cost thresholds, over-utilised resources and potential inefficiencies.
4. Offer filters and drill-down by resource type, cloud provider and environment (production, staging, development).

1.4 Scope

In scope: cost and utilisation ingestion from the three providers (with a realistic mock mode for development and demonstration), a PostgreSQL history store, analytics and recommendations, a role-based web dashboard, an alerting pipeline to Slack and Teams, Docker packaging, and EC2 staging deployment. Out of scope: automatic remediation (for example stopping instances), invoice reconciliation, and multi-tenant SaaS operation. These are listed as future enhancements.

1.5 Report organisation

Chapter 2 reviews existing tools. Chapter 3 lists requirements. Chapters 4 and 5 describe the architecture and design. Chapter 6 covers implementation module by module. Chapter 7 describes alerting, Chapter 8 cost optimisation, Chapter 9 testing, and Chapter 10 deployment. Chapter 11 presents results with screenshots, and Chapter 12 concludes.

2. Existing Solutions and Gap Analysis

Tool	Strengths	Limitations for this problem
AWS Cost Explorer / Budgets	Detailed AWS costs, forecasts	AWS only; alerting limited to budgets
Azure Cost Management	Azure + limited AWS view	AWS connector deprecated; no GCP
GCP Billing reports	Good BigQuery export	GCP only
Commercial FinOps platforms	Multi-cloud, rich features	Licence cost; data leaves the organisation
Grafana + exporters	Flexible charts	No cost normalisation; rules edited as code; no role-aware rule editing

The gap is a **self-hosted, low-cost** tool that normalises all three clouds into one model, adds utilisation context, and lets non-developers define alert rules in a UI, while still using the standard Prometheus/Alertmanager pipeline that DevOps teams already operate.

3. Requirements

3.1 Functional requirements

ID	Requirement
FR-1	Fetch daily costs from AWS Cost Explorer, Azure Cost Management and the GCP billing export
FR-2	Fetch resource inventory and CPU/memory utilisation (CloudWatch, Azure Monitor, Cloud Monitoring)
FR-3	Normalise data into one schema (provider, account, day, service, type, environment, region)
FR-4	Store history in PostgreSQL with scheduled daily and intraday jobs
FR-5	Dashboard with KPIs, trends, breakdowns, utilisation and top resources; auto-refresh
FR-6	Filters by provider, environment, resource type and date range; drill-down to a resource
FR-7	Historical analysis: daily trend, moving average, month-over-month, breakdowns, anomalies
FR-8	User-defined alert rules (metric, scope, threshold, severity, channel)
FR-9	Notifications through Prometheus Alertmanager to Slack and/or Microsoft Teams
FR-10	Login with role-based access (Admin, Viewer)
FR-11	Display amounts in USD or INR with a configurable exchange rate
FR-12	Dark and light theme toggle
FR-13	Switch between mock and live data via .env
FR-14	Monthly reports and CSV export

3.2 Non-functional requirements

Category	Requirement
Performance	Dashboard API responds in < 1 s for 180 days of data
Reliability	Idempotent syncs; one failing provider does not block others; health checks
Security	Hashed passwords, JWT, RBAC enforced by the API, secrets only in .env, read-only cloud permissions
Cost	Minimal paid API calls; runs on a single small instance
Portability	One docker-compose for laptop and EC2
Usability	Responsive layout, accessible contrast, keyboard-usable controls
Maintainability	Modular code, tests, one documentation file per module and screen

4. System Architecture

CloudLens is composed of six containers orchestrated with Docker Compose: PostgreSQL, the FastAPI backend (which also hosts the scheduler and alert engine), the React frontend served by nginx, Prometheus, Alertmanager, and a mock Slack/Teams receiver used in development and demonstrations.

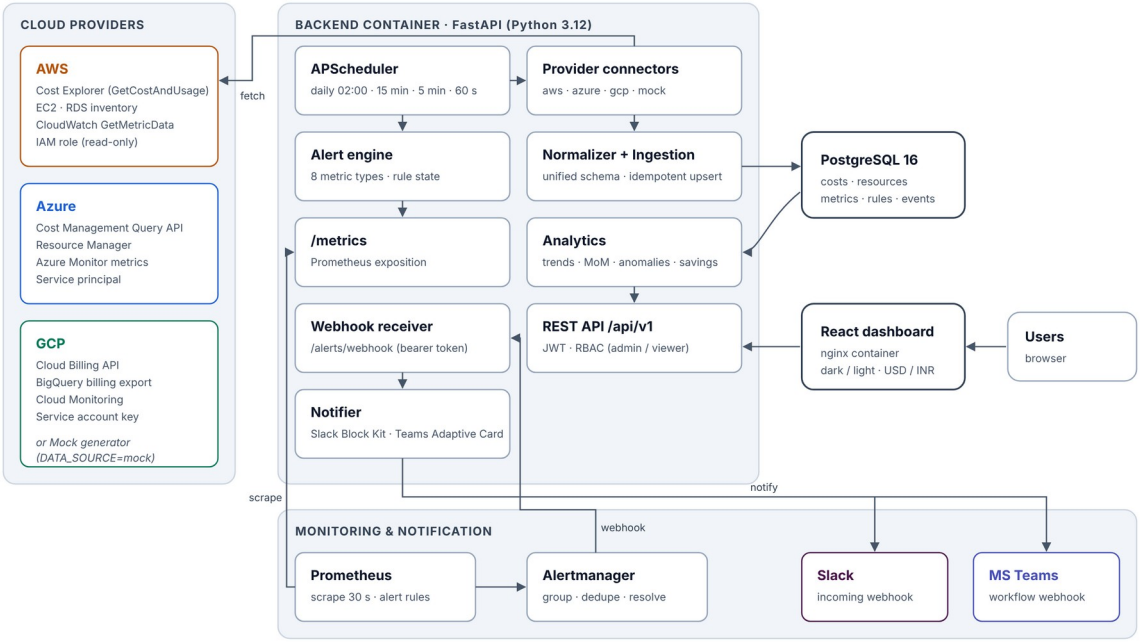


Figure 1: High-level architecture

4.1 Data flow

1. APScheduler triggers jobs: a 180-day backfill on first start, a daily sync at 02:00 IST, an intraday refresh of today's cost every 15 minutes, and utilisation every 5 minutes.
2. The provider factory returns the mock generator or the live connector according to DATA_SOURCE.
3. Connectors return unified CostItem, ResourceItem and MetricItem objects; the normaliser maps service names and tags.
4. The ingestion module upserts the rows with INSERT … ON CONFLICT DO UPDATE on natural keys, so re-runs never duplicate data. Every run is logged with its API-call count.
5. The React app calls the REST API; analytics queries aggregate with indexed GROUP BY queries.
6. The alert engine evaluates rules every minute and exports a Prometheus gauge per rule; Prometheus and Alertmanager handle the alert lifecycle and call the backend webhook, which notifies Slack and Teams.

4.2 Technology choices

Decision	Choice	Reason
Database	PostgreSQL 16	Cost data is tabular and relational; SQL aggregation, unique constraints and composite indexes suit trend analysis (MongoDB was considered)
API	FastAPI	Async-capable, Pydantic validation, auto-generated OpenAPI docs

Decision	Choice	Reason
Scheduler	APScheduler in-process	No message broker needed for a handful of jobs
UI	Custom React + Recharts	Full control of filters, drill-down, RBAC-aware rule editing, theme and currency
Alerting	Prometheus + Alertmanager	Standard tooling: grouping, de-duplication, silences, inhibition
Notifications	Incoming webhooks	Slack Block Kit and Teams Adaptive Cards without bot tokens
Packaging	Docker Compose	Same stack locally and on EC2

5. Design

5.1 Unified cost schema

The key design decision is one grain for every provider: provider + account + day + service + environment + region + resource. The table below shows how each provider's fields are mapped.

Field	AWS	Azure	GCP
usage_date	TimePeriod.Start	UsageDate	DATE(usage_start_time)
service	SERVICE dimension	ServiceName	service.description
environment	tag environment$<v>	TagValue	label environment
region	global	ResourceLocation	location.region
cost_usd	UnblendedCost	Cost (INR→USD)	cost + credits

Service names are classified into compute, containers, database, storage, networking or other by ordered regular-expression rules. Tag values such as prod, prd, stg, uat, dev and test are mapped to production, staging, development or unassigned.

5.2 Database tables

Table	Purpose	Key constraint / index
cost_records	Normalised daily cost lines	UNIQUE on the natural grain; (provider|environment|type, date) indexes
resources	Inventory	UNIQUE(provider, resource_id)
resource_metrics	CPU / memory / storage samples	UNIQUE(resource_pk, ts)
alert_rules	User-defined rules + live state	—
alert_events	Notifications sent and delivery status	index on fingerprint, created_at
sync_runs	Audit of every fetch incl. API calls	index on provider, started_at
users	Accounts and roles	UNIQUE(email)
app_settings	Budget, exchange rate, channel, thresholds	key primary key

5.3 API design

The REST API under /api/v1 is grouped into auth/users, dashboard, costs, resources, alerts, sync, settings and reports. All read endpoints accept the same filter parameters (days or start/end, provider, environment, resource_type, service, region), validated centrally. Write endpoints require the admin role. The Dashboard screen uses one aggregated /dashboard/overview call to minimise round trips. Interactive documentation is generated at /docs.

5.4 Module structure

Layer	Modules
core	config (pydantic-settings), database (SQLAlchemy), security (bcrypt, JWT)
services	providers (aws, azure, gcp, mock), normalizer, ingestion, analytics, alert_engine, prom_metrics, notifier, scheduler, settings_service, seed

Layer	Modules
api	deps (auth, RBAC, filters), routes (8 routers)
frontend	api client, Auth and Prefs contexts, useApi hook, shared components, charts, 10 pages

6. Implementation

The work followed the six-sprint plan in content but was delivered in a single pass. This chapter describes the implementation in the order of the sprint goals.

6.1 Cloud integration (Sprint 1)

- **AWS:** boto3 Cost Explorer GetCostAndUsage (DAILY, UnblendedCost, grouped by SERVICE and the environment tag, paginated), EC2/RDS inventory, CloudWatch GetMetricData. It supports access keys, profiles, or the EC2 instance role.
- **Azure:** Cost Management Query API (ActualCost, daily, grouped by service, location and environment tag) in 7-day windows, Resource Manager inventory, and Azure Monitor CPU metrics, using a service principal.
- **GCP:** the Cloud Billing API does not return daily cost lines, so costs are read from the BigQuery billing export with partition filters and a 2 GB scan guard; the Cloud Billing API verifies the billing account; Cloud Monitoring supplies instances and CPU.
- **Mock mode:** a deterministic generator (hash of resource and date) produces 25 realistic resources with growth, weekly seasonality, noise, periodic anomalies and deliberate waste, so the system can be developed and demonstrated without credentials. DATA_SOURCE=mock|live switches modes.
- Credentials are configured in .env; the Cloud Accounts screen offers a Test connection button for each provider.

6.2 Aggregation and storage (Sprint 2)

The ingestion module batches upserts in chunks of 1000 rows, merging duplicates in memory first because ON CONFLICT cannot update a row twice in one statement. Each provider is synchronised independently; a provider without credentials is recorded as skipped, and an exception is recorded as failed without affecting the others. A retention job removes old metric samples (90 days), cost lines (2 years) and sync logs (60 days).

6.3 Real-time dashboard (Sprint 3)

The React dashboard implements the design approved before development: five KPI cards, a daily cost trend with a daily-budget line and anomaly markers, spend by provider (donut) and by resource type, current utilisation per provider, recent alerts, and the top resources table. Utilisation is sampled every 5 minutes and today's cost every 15 minutes; the page refreshes every 60 seconds. A shared filter bar (provider, environment, resource type, date range) is kept in session storage so it follows the user between screens.

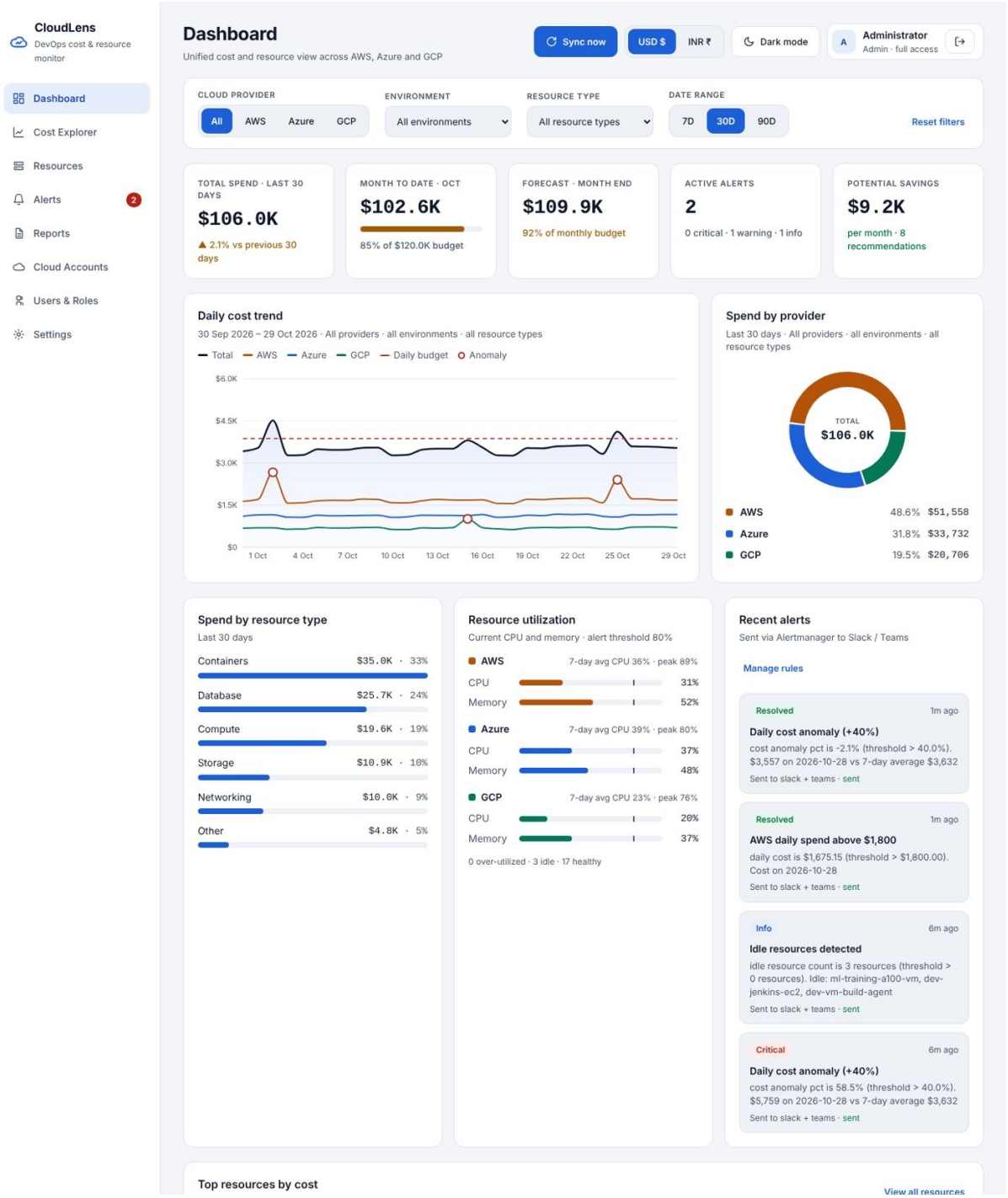


Figure 2: Dashboard (light theme)

6.4 Historical analysis (Sprint 4)

Analytics functions compute zero-filled daily series with a 7-day moving average, breakdowns by any dimension compared with the previous equal-length period, monthly totals with month-over-month change (and run-rate change for the partial current month), forecasts (full days so far + 7-day average × remaining days), and anomalies (days more than 40 % above the trailing 7-day average). The Cost Explorer screen adds custom date ranges, group-by switching, 3/6/12-month views and CSV export.

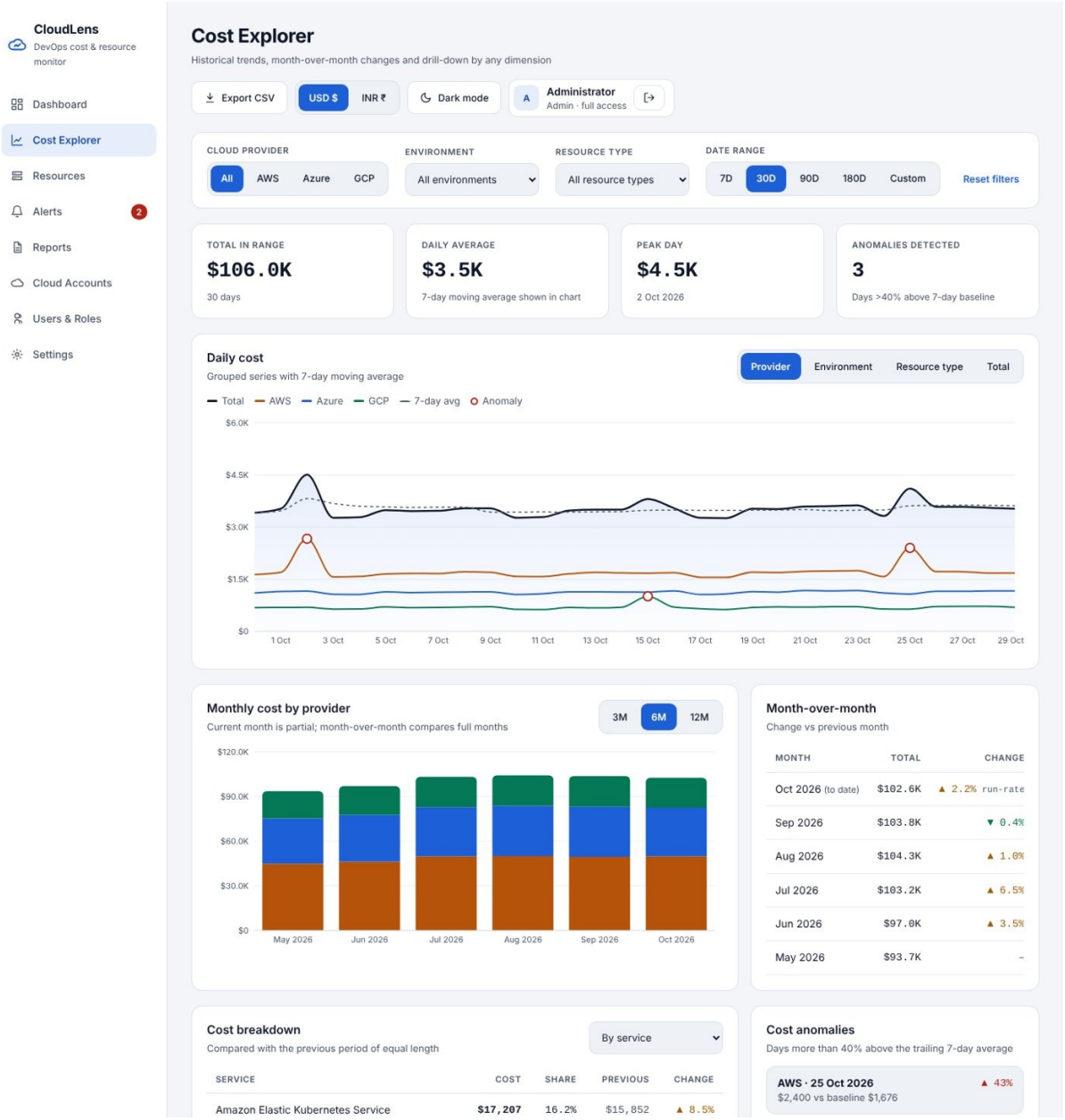


Figure 3: Cost Explorer with historical trends and month-over-month table

6.5 Resources and drill-down

The Resources screen lists every resource with period cost, latest and 7-day CPU, and a status: over-utilised (CPU ≥ 80 %), idle (7-day average below 10 %), healthy, or no metrics. Selecting a row opens the resource detail page with 30 days of cost, 7 days of utilisation, tags and any recommendation.

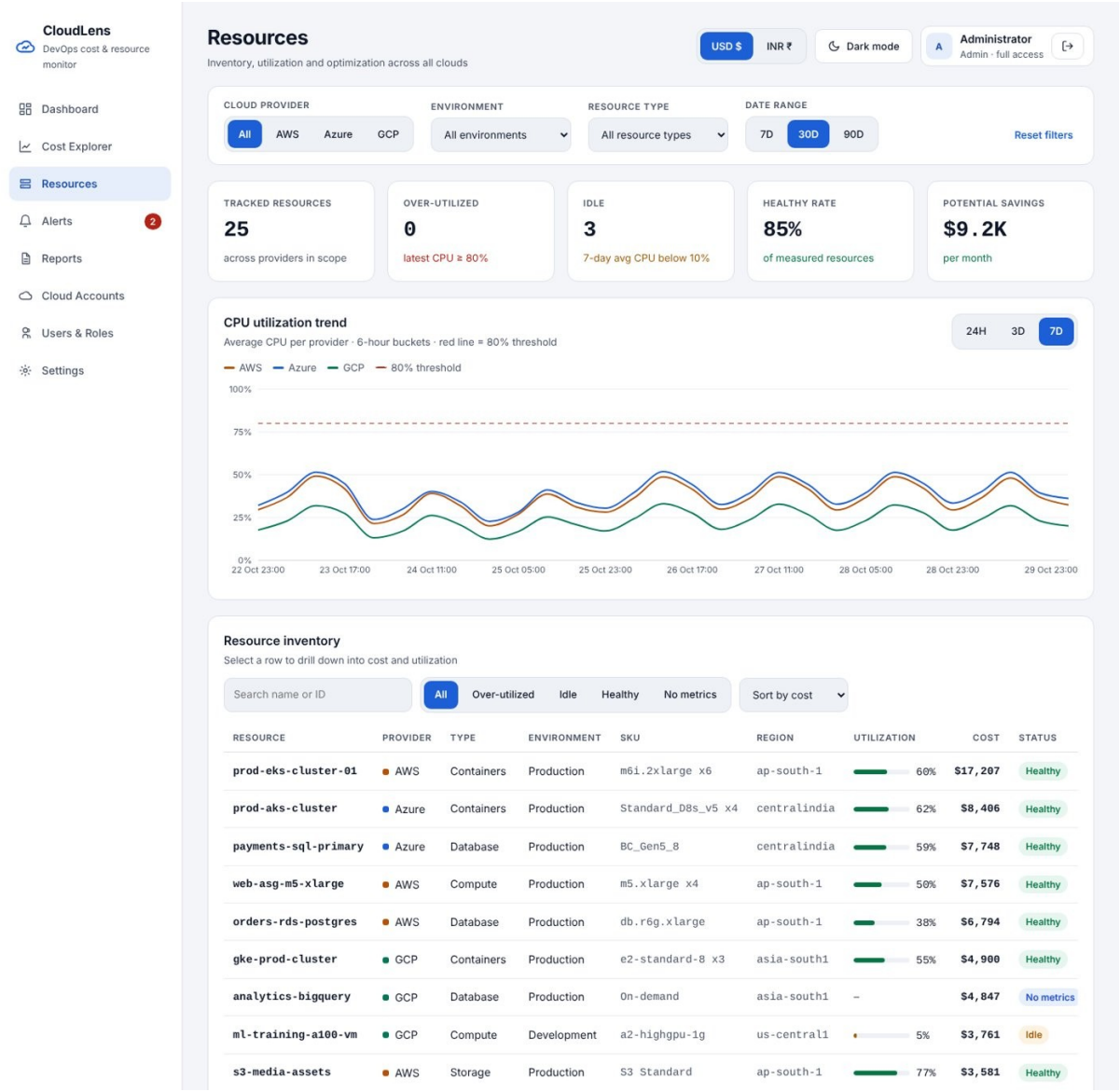


Figure 4: Resources: utilisation trend, inventory and recommendations

6.6 Authentication and roles

Passwords are hashed with bcrypt and sessions use signed JWTs (8-hour expiry). FastAPI dependencies enforce authentication on every route and the admin role on every write. The UI hides admin-only navigation and actions for viewers, but the API is the source of truth. Admins cannot demote or delete themselves, which prevents lock-out.

6.7 Currency and theme

The API always returns USD; the frontend converts to INR with the exchange rate stored in Settings and formats Indian units (K, lakh, crore). Themes are implemented with CSS custom properties for dark and light; charts use the same tokens, so switching is instant and contrast stays above WCAG AA in both themes.

7. Alerting and Notifications (Sprint 5)

Users define rules in the Alerts screen. Every minute the alert engine computes each rule's value and exports cloudlens_alert_rule_breached with the rule's labels. A single Prometheus rule (CloudLensRuleBreached, for: 1m) turns breached series into alerts; Alertmanager groups, de-duplicates, repeats every 4 hours and sends resolved notifications to the backend webhook (protected by a bearer token). The backend stores an AlertEvent and posts a Slack Block Kit message and/or a Teams Adaptive Card, according to the rule's channel or the default preference.



Figure 5: Alert pipeline

Metric	Meaning	Example rule
daily_cost	Cost of the last full day	AWS > $1,800 (warning)
mtd_cost	Month-to-date cost	Development > $5,000
budget_used_pct	MTD as % of budget	≥ 80 % (warning)
forecast_budget_pct	Forecast as % of budget	> 100 % (critical)
cost_anomaly_pct	Last day vs 7-day average	> 40 % (critical)
resource_cpu / resource_memory	Highest current utilisation in scope	Production CPU > 80 %
idle_resource_count	Resources with 7-day avg CPU < 10 %	> 0 (info)

Platform alerts are included too: backend down (routed straight to Slack, because the backend cannot relay it), sync failing, stale data and high API latency, with an inhibition rule to avoid duplicate noise. A mock receiver stands in for Slack and Teams during development and the viva demonstration, and an admin-only "simulate a cost spike" action makes the anomaly path easy to demonstrate.

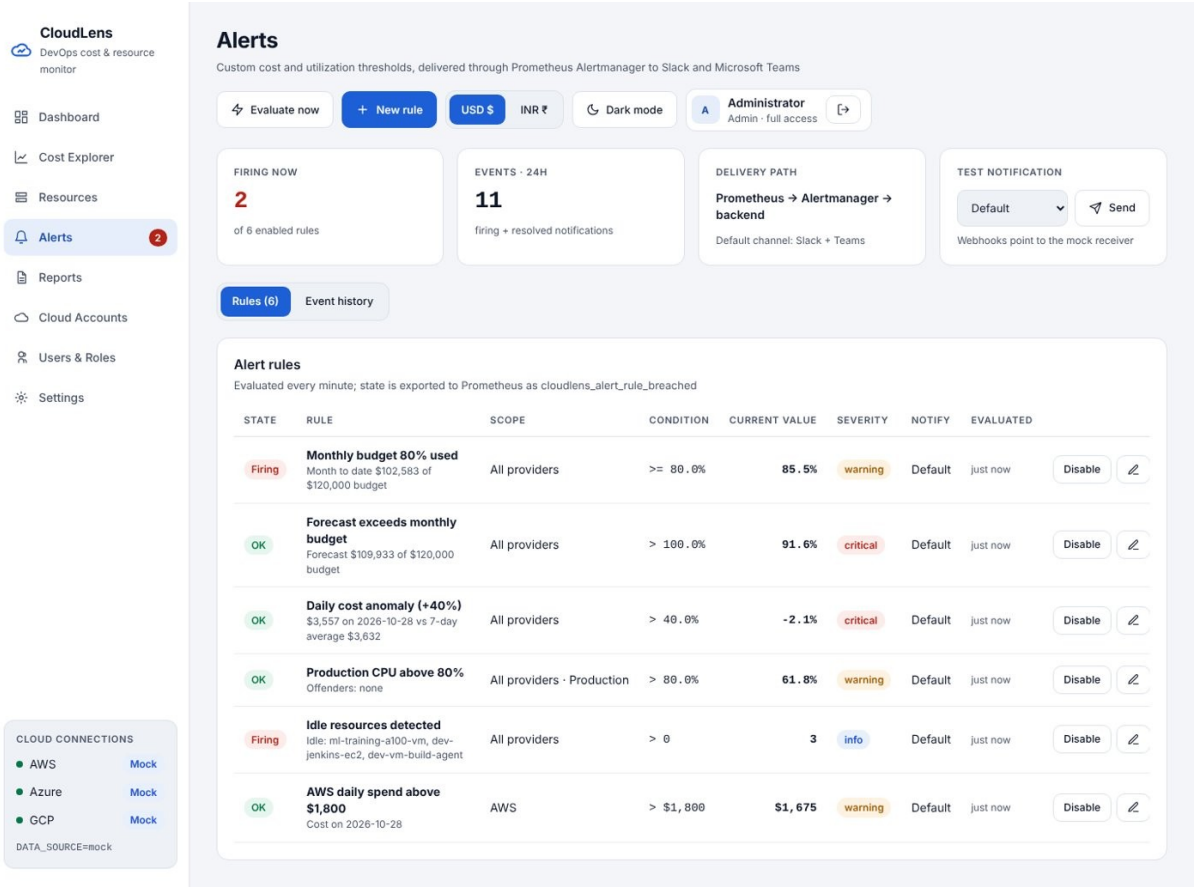


Figure 6: Alerts screen: rules with live state

8. Cost Optimisation

8.1 Savings for the monitored clouds

Rule	Condition	Action	Est. saving
Idle (non-prod)	avg CPU < 10 %	Stop / schedule off	90 %
Idle (prod)	avg CPU < 10 %	Rightsize	50 %
Under-utilised	CPU < 30 % and mem < 50 %	One size down	40 %
Non-prod 24x7	dev/staging compute	Nights/weekends off	60 %
Low storage use	< 20 % used	Archive tier / lifecycle	45 %

On the demonstration data the engine finds 8 recommendations worth about USD 9.2K per month, led by an idle GPU training VM in development. Budget and forecast alerts catch overruns mid-month, and anomaly detection flags runaway jobs within a day.

8.2 Keeping the platform itself cheap

- Cost Explorer charges USD 0.01 per request, so the daily job re-reads only 3 days, intraday refresh is configurable (6 hours recommended live), and every sync records its API calls: about 2 requests a day (≈ USD 0.60/month) instead of about USD 30 for minute-level polling.
- BigQuery queries filter on the partition column with a 2 GB maximum-bytes-billed guard, keeping usage within the free tier.
- The whole stack runs on one t3.small (single backend worker, in-process scheduler, no Redis), with retention policies, 15-day Prometheus retention and Docker log rotation that keep the disk under 2 GB.
- A multi-stage nginx frontend image (≈ 50 MB), an optional cloud-SDK-free backend image, and an IAM instance role instead of long-lived keys.

Staging item	USD / month
t3.small on-demand	~15.2
20 GB gp3	~1.9
Public IPv4	~3.6
Cost Explorer API	~0.6
Total	≈ 21 (₹1,770)

9. Testing

Suite	Tests	Result
Backend on SQLite	77	77 passed
Backend on PostgreSQL 16	77	77 passed
Backend with pinned requirements (clean venv)	77	77 passed
Frontend (Vitest)	9	9 passed
promtool / amtool / docker compose config	—	valid
End-to-end alert pipeline	5 steps	passed

Backend line coverage is 85 % (analytics 96 %, alert engine 94 %). Key automated checks: breakdowns by provider, environment, type and service always add up to the period total; re-syncing never changes row counts; viewers receive HTTP 403 on every write endpoint; an injected cost spike is detected as an anomaly; Alertmanager duplicates are ignored; and Slack and Teams payloads have the expected structure.

End-to-end, real Prometheus 2.55.1 and Alertmanager 0.27.0 were run against the backend: an injected AWS cost spike caused four rules to fire, eight messages (Slack and Teams) were delivered, and clearing the spike produced RESOLVED messages. Every screen was captured with Playwright in both themes and at phone width, with no application console errors.

ID	UI test case	Expected
UI-02	Sign in as viewer	Admin menus and actions hidden; Settings read-only
UI-04	Switch to INR	Amounts in ₹ with lakh/crore units
UI-05	Filter AWS	All widgets limited to AWS; AWS budget used
UI-08	Click a resource	Drill-down with cost, utilisation and tags
UI-09	Create rule	Evaluated immediately; state shown
UI-10	Test notification	Message delivered; event marked sent
UI-14	Phone width	Sidebar stacks, cards wrap, tables scroll

10. Deployment

The same docker-compose.yml runs locally and on AWS EC2. Only the dashboard port is public; the API, Prometheus, Alertmanager and the mock receiver are bound to localhost and reached through nginx or an SSH tunnel. Containers run as non-root users with health checks and log rotation, and data persists in named volumes.

1. `create-ec2.sh` creates a security group (SSH from the operator IP only, HTTP 80), an IAM role with a read-only policy, and a t3.small Amazon Linux 2023 instance with IMDSv2 and user data that installs Docker and swap.
2. `.env.staging` holds the secrets (database password, JWT secret, webhook URLs) and AWS_USE_DEFAULT_CREDENTIALS=true to use the instance role.
3. `deploy.sh` synchronises the project, uploads the environment file, runs docker compose up --build, and waits for /health.

```
cp .env.example .env.staging   # edit secrets
KEY_NAME=my-key ./deploy/aws/create-ec2.sh
EC2_HOST=<ip> SSH_KEY=~/.ssh/my-key.pem ./deploy/aws/deploy.sh
```

A database backup script, a Makefile of common tasks, and a production-hardening checklist (HTTPS, removing the mock receiver, rotating defaults, EBS snapshots) are provided.

11. Results

All four project goals were met. The screens below show the delivered application with mock data.

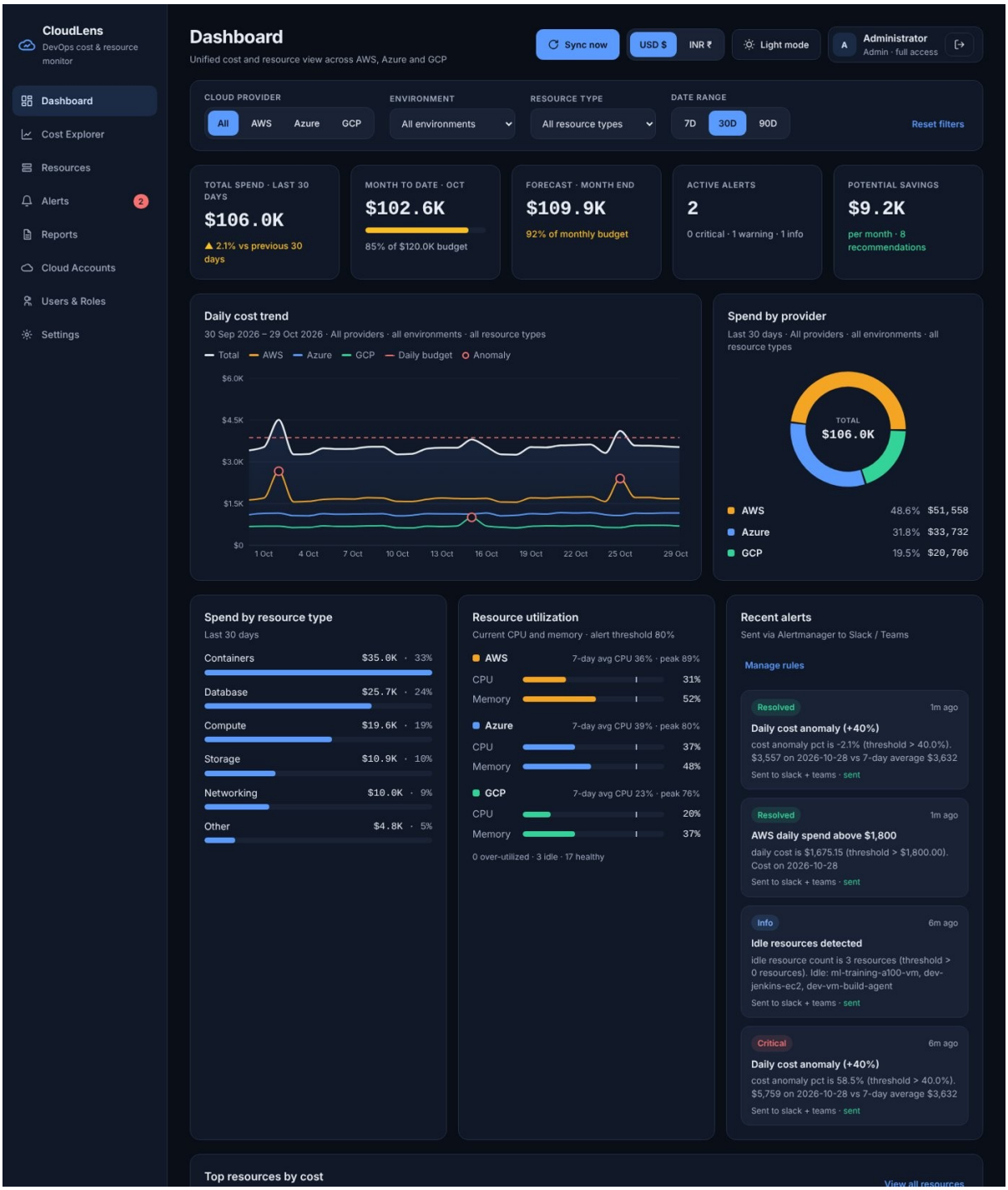


Figure 7: Dashboard in dark theme (approved design)

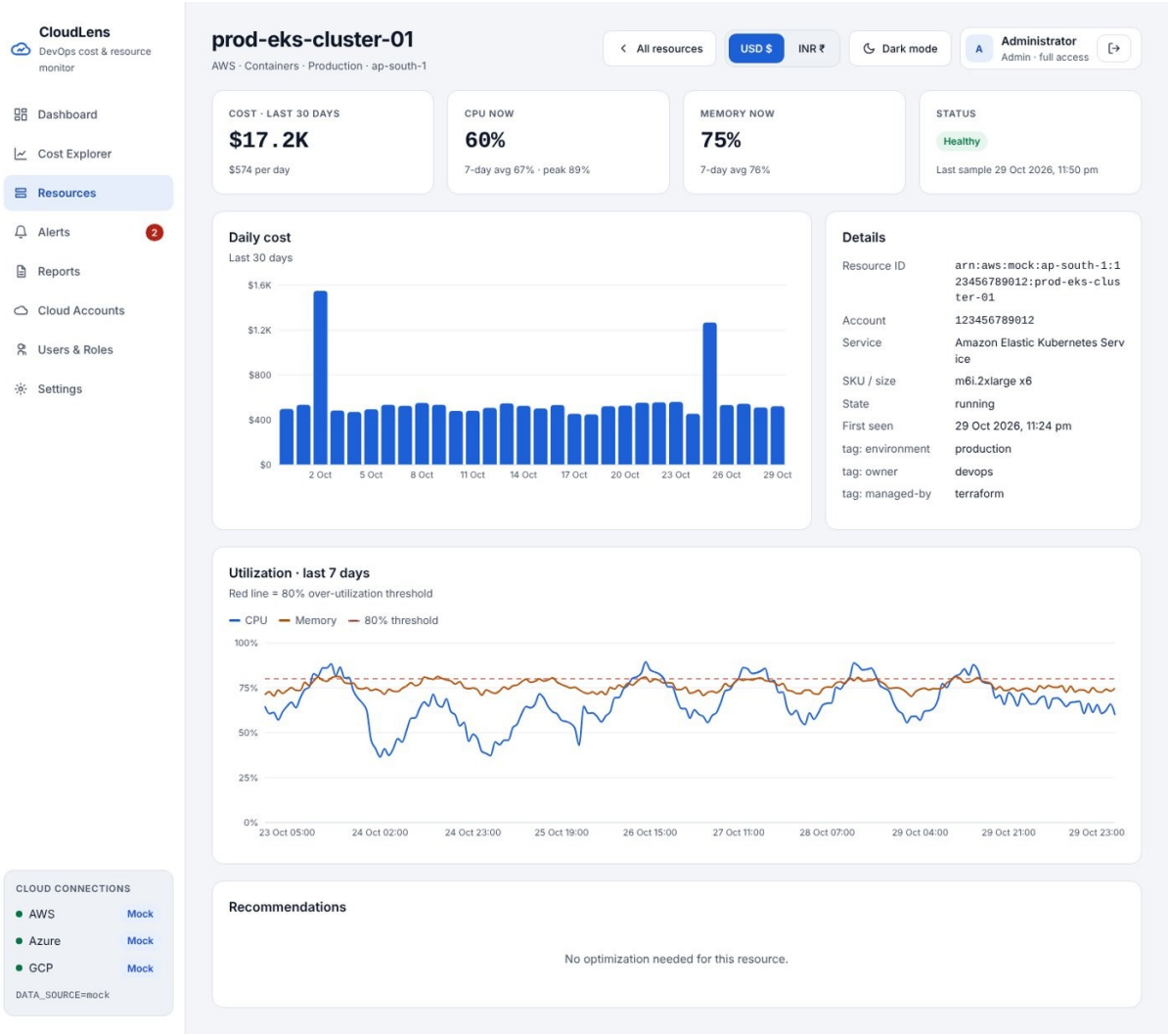


Figure 8: Resource drill-down

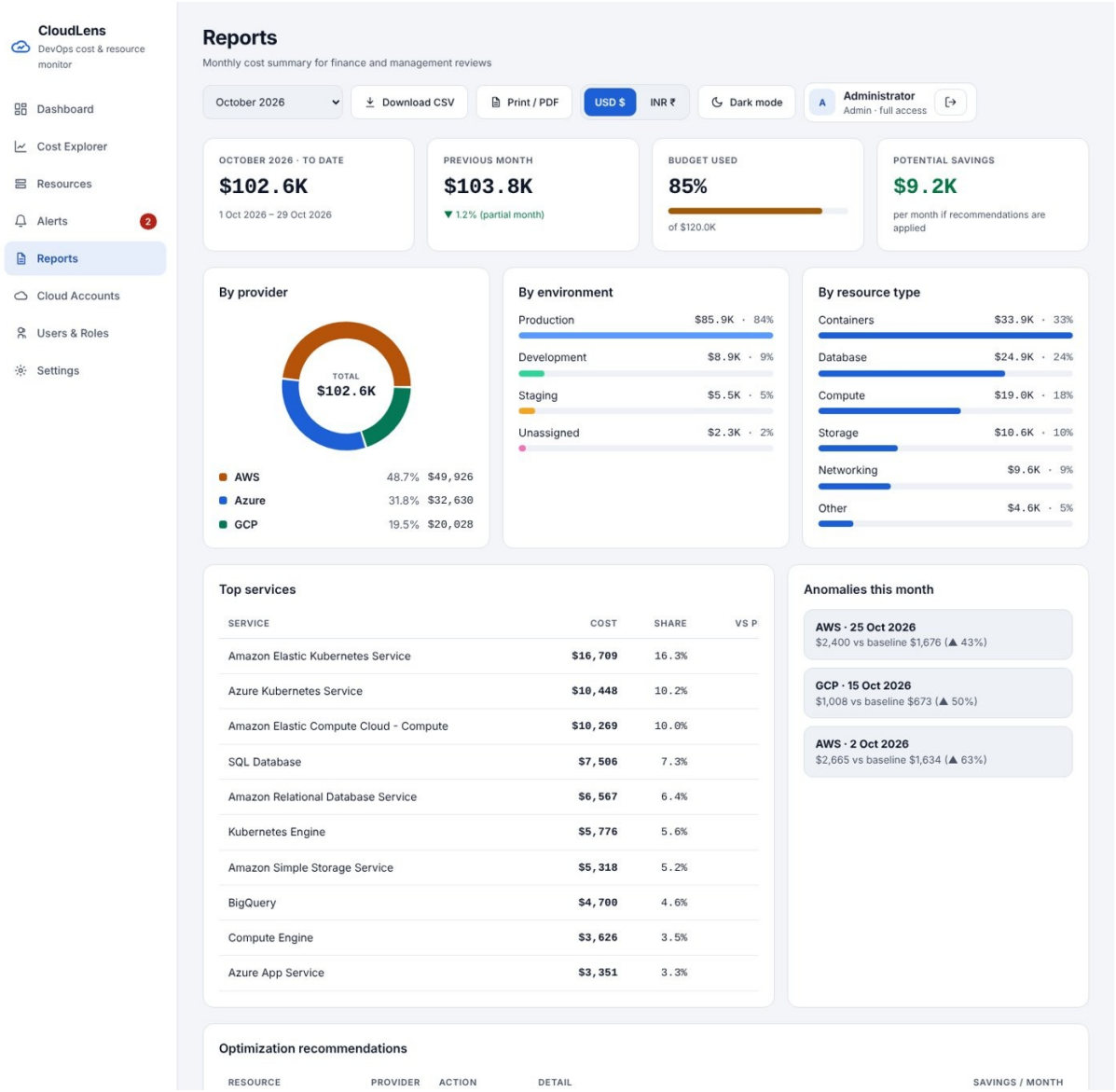


Figure 9: Monthly report

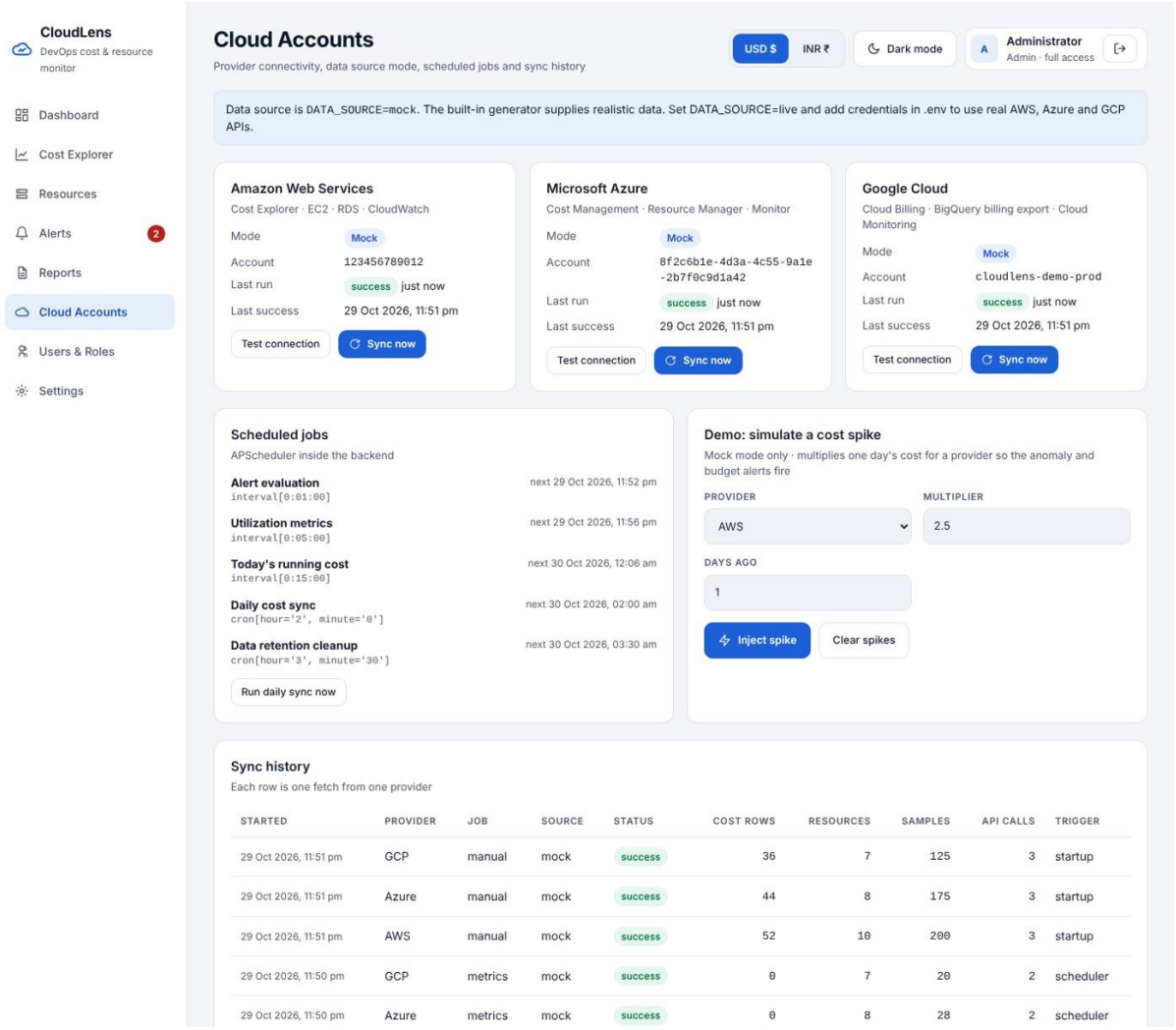


Figure 10: Cloud Accounts: connectivity, scheduled jobs, spike simulation, sync history

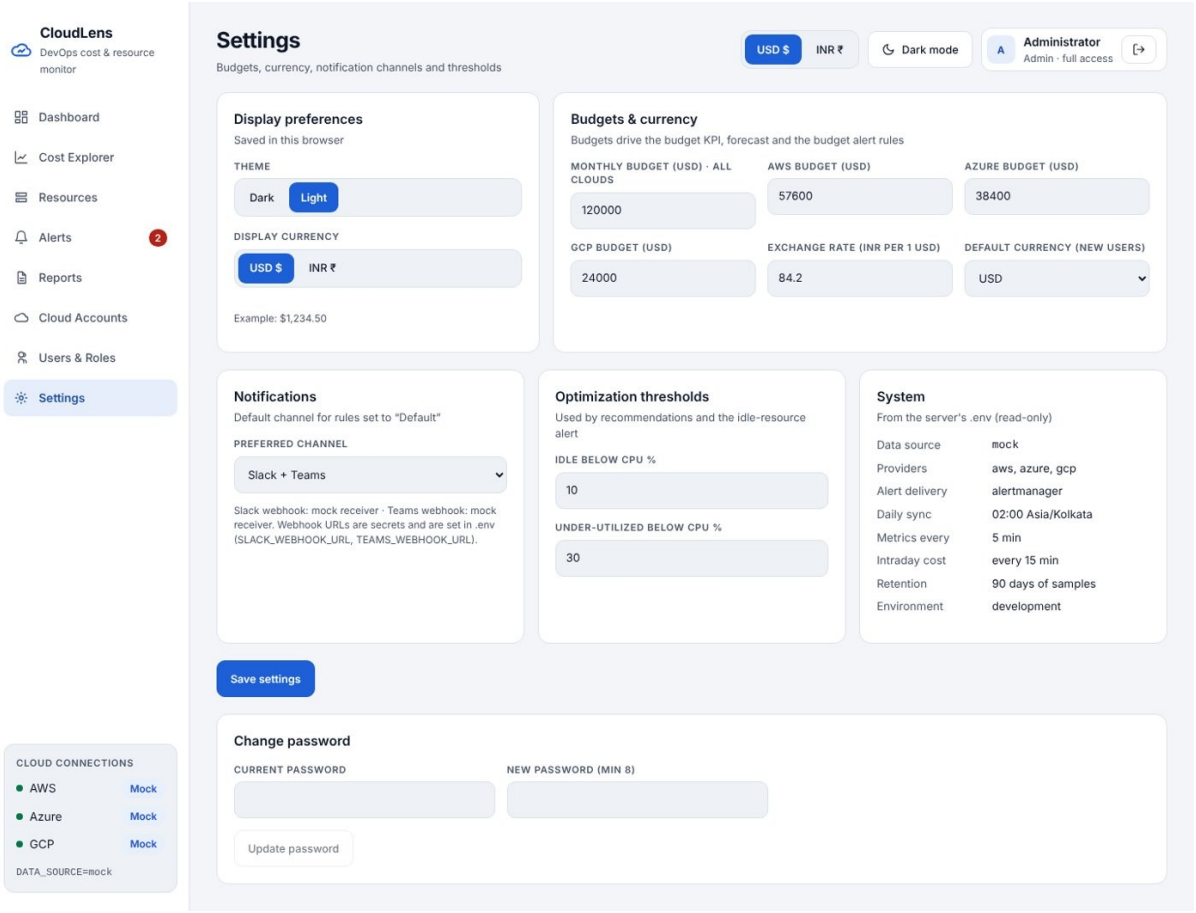


Figure 11: Settings: budgets, exchange rate, channels, thresholds

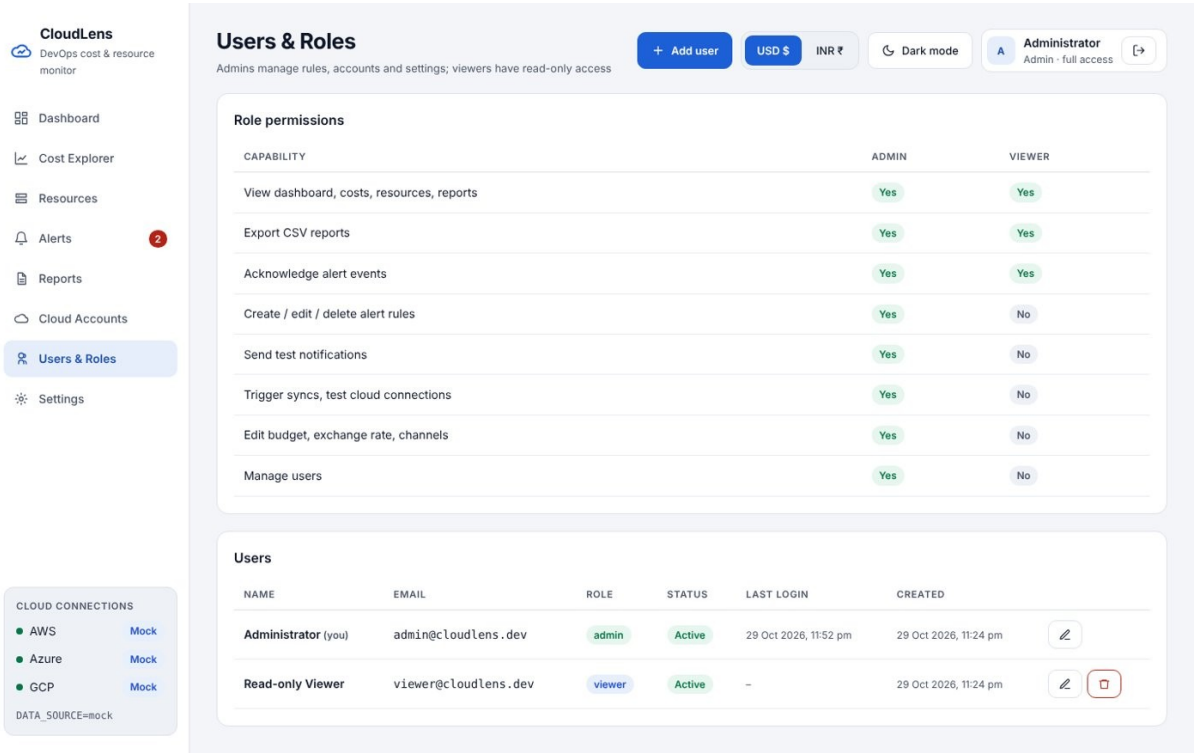


Figure 12: Users & Roles

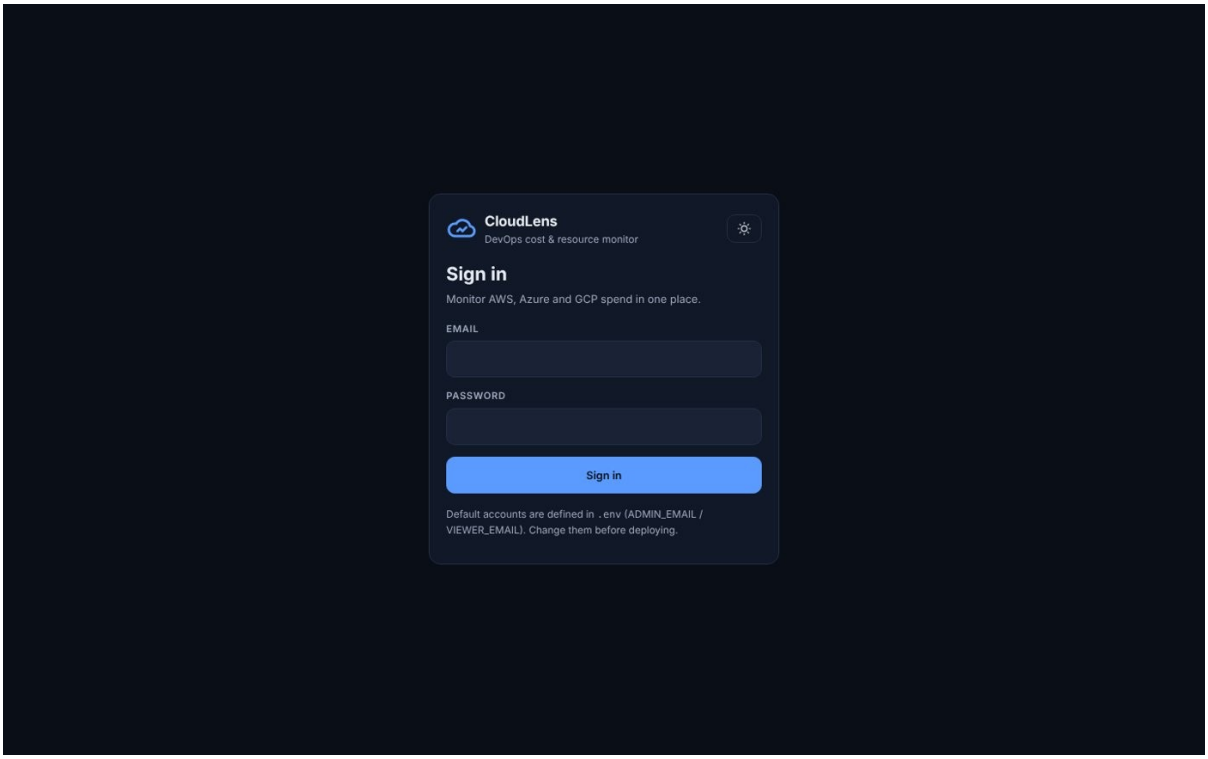


Figure 13: Login

Goal	Outcome
Aggregate AWS, Azure, GCP	Three connectors + mock; unified schema; 180-day backfill in ~3 s
Real-time and historical visualisation	5-min utilisation, 15-min cost, 60 s refresh; trends, MoM, anomalies, reports
Customisable alerting	8 metric types; Prometheus → Alertmanager → Slack/Teams verified end to end
Filters and drill-down	Provider, environment, type, date range; per-resource drill-down

12. Conclusion and Future Scope

12.1 Conclusion

CloudLens shows that a small, self-hosted platform can give DevOps teams a single, trustworthy view of multi-cloud cost and utilisation. Normalising the three providers into one schema made cross-cloud comparison and filtering straightforward. Bridging user-defined rules into Prometheus kept the standard alerting toolchain while making rules editable by non-developers. The recommendations engine turns monitoring into measurable savings, and the platform's own running cost stays around USD 21 per month.

12.2 Learnings

- Each cloud exposes cost data differently: GCP requires the BigQuery export, AWS limits Cost Explorer to two group-by keys, and Azure pages large results. A unified data contract hides these differences from the rest of the system.
- Idempotent upserts and per-provider error isolation make scheduled ingestion safe to re-run.
- Paid monitoring APIs need cost-aware design: sync windows, caching and call accounting.

12.3 Future scope

- Alembic migrations, HTTPS, and a live exchange-rate feed.
- A separate worker container so the API can scale; table partitioning or TimescaleDB.
- ML-based forecasting and per-service anomaly detection.
- SSO (Azure AD / Google), audit log, scheduled email reports, Kubernetes cost allocation.
- Automated remediation with approval workflows; Savings Plan / RI recommendations.

References

1. AWS Cost Explorer API Reference: GetCostAndUsage. Amazon Web Services documentation.
2. Azure Cost Management Query API. Microsoft Learn.
3. Export Cloud Billing data to BigQuery. Google Cloud documentation.
4. Prometheus alerting rules and Alertmanager configuration. prometheus.io documentation.
5. Slack incoming webhooks and Block Kit. Slack API documentation.
6. Microsoft Teams Workflows webhooks and Adaptive Cards. Microsoft Learn.
7. FastAPI, SQLAlchemy 2.0, APScheduler, React, Recharts project documentation.
8. FinOps Foundation: FinOps Framework.

Appendix A: Repository and Documentation Map

Path	Contents
backend/	FastAPI service, connectors, analytics, alert engine, tests
frontend/	React dashboard
monitoring/	Prometheus config and rules, Alertmanager template, mock receiver
deploy/aws/	EC2 creation, bootstrap, deploy scripts, IAM policy
docs/	Architecture, setup, user guide, API, database, alerting, deployment, testing, cost optimisation, troubleshooting, sprint mapping
docs/modules/	One document per module (15)
docs/screens/	One document per screen (10)

Appendix B: Key Configuration

Variable	Default	Meaning
DATA_SOURCE	mock	mock or live
ENABLED_PROVIDERS	aws,azure,gcp	Providers to sync
BACKFILL_DAYS	180	History on first start
METRICS_INTERVAL_MINUTES	5	Utilisation sampling
INTRADAY_COST_REFRESH_MINUTES	15	Today's cost refresh (360+ live)
ALERT_DELIVERY	alertmanager	or direct
SLACK_WEBHOOK_URL / TEAMS_WEBHOOK_URL	mock receiver	Notification endpoints
DEFAULT_MONTHLY_BUDGET_USD	120000	Initial budget
DEFAULT_EXCHANGE_RATE_INR	84.20	Initial INR rate